Documento técnico

ETL de siniestralidad vial y cámaras de tránsito de Medellín

Cómo está construido hoy el proceso de extracción, depuración y carga, qué ruido corrige y cómo se opera. Versión del 6 de noviembre de 2026 · Equipo IA 02, Cementos Argos.

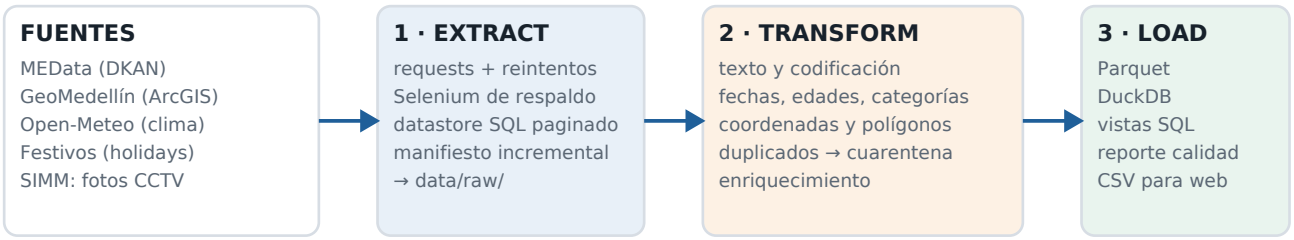


Figura 1. Flujo general. Cada etapa se puede ejecutar por separado desde la línea de comandos.

1. Resumen

El proceso descarga de forma automática los datos abiertos de siniestralidad vial del Distrito de Medellín, las capas georreferenciadas de cámaras y semáforos, y datos de contexto (clima y festivos). Limpia el ruido que traen las fuentes, enriquece cada registro con su comuna, la cámara más cercana y la lluvia de esa hora, y guarda el resultado en Parquet y en una bodega DuckDB lista para Power BI o SQL.

Aspecto	Decisión
Alcance geográfico	Solo el Distrito de Medellín: 16 comunas y 5 corregimientos, validado contra el polígono oficial.
Lenguaje	Python 3.11+ (pandas, requests, shapely, DuckDB, pyarrow).
Extracción	APIs oficiales con requests; Selenium solo como respaldo si cambia la API de MEData.
Almacenamiento	data/raw (copia fiel) → data/clean (Parquet) → data/warehouse (DuckDB).
Trazabilidad	Nada se borra en silencio: cuarentena con motivo, banderas por fila y reporte de calidad.
Pruebas	16 pruebas automáticas con muestras reales de las fuentes (pytest).

2. Fuentes de datos

Fuente	Contenido	Acceso	Periodo
MEData · Víctimas en incidentes viales (1-023-25-000360)	Una fila por víctima: gravedad, edad, sexo, condición, coordenadas. Dataset principal.	API DKAN → CSV (54 MB)	2014-2021
MEData · Incidentes viales (1-023-25-000094)	Una fila por incidente: clase, gravedad, diseño de vía. Se une a víctimas por radicado.	API DKAN → CSV (62 MB)	2014-2020
GeoMedellín · Siniestralidad (VM_Accidentes/0)	Siniestros georreferenciados con heridos y solo daños, actualizados de forma constante.	ArcGIS REST, páginas de 1.000	2019-2025
MEData · Aforos, velocidades, incidentes con moto	Exposición: volumen y velocidad por corredor.	API DKAN → CSV	varía
GeoMedellín · CCTV_Movilidad	81 cámaras de circuito cerrado con enlace a la foto en vivo.	ArcGIS REST	vigente
GeoMedellín · Camaras_FDT	50 cámaras de fotodetección: infracciones que detecta y velocidad máxima.	ArcGIS REST	vigente
GeoMedellín · Camaras_ARS	67 cámaras de apoyo a la red semafórica.	ArcGIS REST	vigente
GeoMedellín · Cruce semafórico	724 cruces semaforizados.	ArcGIS REST	vigente
GeoMedellín · Comunas y barrios	Polígonos oficiales para validar y asignar comuna.	ArcGIS REST (simplificado)	vigente
SIMM · Cámaras CCTV	Imagen JPG 1280×720 por cámara, se refresca en el servidor.	HTTP (imagen{N}.jpg)	en vivo
Open-Meteo Archive (ERA5)	Lluvia, temperatura, humedad y visibilidad por hora.	API sin llave	2014-hoy
Librería holidays	Festivos de Colombia (ley Emiliani).	local	—

Los tableros del Observatorio de Movilidad están en Power BI embebido, que no permite descargas programáticas estables. Por eso el proceso va directo a las fuentes abiertas que alimentan esos tableros.

3. Etapa 1 · Extract

Módulo **etl/extract.py**. Cada fuente tiene un cliente con sesión HTTP compartida: 4 reintentos con espera exponencial ante errores 429/5xx y tiempo límite configurable.

3.1 MEData (DKAN)

- Consulta `/api/dataset-fast/{id}` y toma la distribución CSV (`downloadURL`), forzada a HTTPS.
- Si esa API falla, abre la página del dataset con Chrome headless (Selenium) y lee el enlace .csv del DOM, ya que la página es una aplicación React.
- Si la descarga directa falla, recorre el datastore SQL de DKAN con `LIMIT/OFFSET` de 500 filas.
- Descarga en streaming a un archivo .part que se renombra al terminar, para no dejar archivos a medias.

3.2 GeoMedellín (ArcGIS REST)

- Consulta `/query?where=1=1&outFields=*&outSR=4326&f=geojson` paginando con resultOffset hasta agotar la capa.
- Los polígonos se piden simplificados (maxAllowableOffset) para que pesen poco.

3.3 Clima

- Un archivo por año desde 2014. Los años cerrados no se vuelven a pedir; el año en curso se refresca con 6 días de rezago (ERA5).

3.4 Extracción incremental

El archivo `data/raw/_manifest.json` guarda por fuente la fecha modified del catálogo, la URL, el tamaño, el ETag y la hora de extracción. Si nada cambió, la descarga se omite. `--force` obliga a descargar de nuevo.

4. Etapa 2 · Transform

Módulos **etl/cleaning.py** (funciones reutilizables), **etl/geo.py** (polígonos) y **etl/transform.py** (reglas por fuente). Tres principios:

- Lo inservible (filas vacías, duplicados, fechas imposibles) va a `data/quarantine/` con su motivo_rechazo.
- Lo corregible se corrige y queda anotado en la columna `flags_calidad` de la fila.
- Cada regla registra cuántas filas tocó en `data/reports/calidad_datos.json` y `.md`.

4.1 Ruido encontrado al perfilar los archivos completos

Fuente	Problema	Filas	Acción
Víctimas	CSV rellenado con filas vacías hasta el límite de Excel (1.048.576)	812.475	Eliminadas
Víctimas	Comuna «Sin Inf»	21.726	Imputada por polígono si hay coordenada
Víctimas	Sin coordenadas	20.845	Se conservan con bandera
Víctimas	Coordenada fuera de Medellín o lat/lon invertidas	2.116	Invertidas se corrigen; el resto se anula
Víctimas	Edad «Sin Inf», vacía, 0 o mayor de 100	3.753	Nulo y grupo de edad recalculado
Víctimas	Sexo «Sin Inf» / «Sin inf»	2.477	Nulo
Víctimas	Mayúsculas distintas en condición	7.257	Unificado
Víctimas	Duplicados exactos	197	Cuarentena
Incidentes	Año con \r literal pegado («2019\r»)	26.760	Se extraen los dígitos
Incidentes	Texto con escapes hexadecimales («Solo da\xF1os»)	20.560	Decodificado
Incidentes	Comuna vacía, 0, In, AU, No Georef o Sin Inf	19.861	Nulo e imputada por polígono
Incidentes	Coordenada comodín [-75,7037; 6,2214] usada cuando no se geocodificó	19.613	Anulada
Incidentes	Variantes de «Caída de ocupante»	6.527	Unificado
Incidentes	Radicado repetido	34	Cuarentena
Cámaras	CCTV sin enlace de foto o con enlace roto («imagen.jpg»)	11	Se conservan sin foto
Cámaras	ARS con geometría vacía y nombre en blanco	2	Descartadas

Cifras obtenidas al perfilar los archivos completos publicados en MEData y GeoMedellín en noviembre de 2026.

4.2 Reglas de limpieza, en orden de ejecución

#	Regla	Detalle
1	Filas vacías	Se eliminan las filas sin ningún valor.
2	Texto	Repara mojibake (MoscÃº → Moscú) con ftfy, decodifica escapes \xNN, quita \r y espacios dobles.
3	Duplicados	Exactos y por radicado → cuarentena.
4	Fecha y hora	Se combinan en fecha_hora; fechas imposibles o futuras → cuarentena; se marca si el año declarado no coincide.
5	Categorías	Diccionarios canónicos para gravedad, clase, condición y diseño de vía; se agrega actor_vial (moto, peatón, bicicleta, vehículo).
6	Edad y sexo	Edad válida de 1 a 99; sexo solo M o F; lo demás es nulo.
7	Coordenadas	Coma decimal a punto; lat/lon invertidas se corrigen; comodín y fuera de la caja se anulan.
8	Validación espacial	Punto en polígono contra comunas oficiales: imputa comuna faltante, marca inconsistencias y anula puntos en municipios vecinos.
9	Fuera de Medellín	Sin comuna declarada y con punto fuera del Distrito → cuarentena.

4.3 Enriquecimiento

Variable	Cómo se calcula
Calendario	año, mes, hora, día de la semana, fin de semana, festivo, día hábil, franja horaria y hora pico (6-8 y 17-19 en días hábiles).
Comuna geográfica	codigo_comuna_geo y comuna_geo por punto en polígono (índice espacial STRtree).
Cámara más cercana	Distancia haversine vectorizada a la cámara CCTV, FDT y ARS más cercana; id de esa cámara y si está a 300 m o menos.
Clima	Lluvia (mm), si llovía (≥ 0,2 mm), temperatura, humedad y visibilidad de la hora del incidente.
Dimensión fecha	Festivo y temporada: Navidad y fin de año, vacaciones de mitad de año, Feria de las Flores.

5. Etapa 3 · Load

Módulo **etl/load.py**. Cada tabla se escribe en Parquet y se carga en DuckDB con `CREATE OR REPLACE`, así una corrida nueva reemplaza la anterior sin duplicar.

Tabla	Grano	Contenido principal
fact_victima	víctima	fecha_hora, gravedad, actor vial, edad, sexo, comuna, coordenadas, cámara cercana, lluvia, banderas
fact_incidente	incidente (MEData)	clase, gravedad, diseño de vía y el mismo contexto
fact_siniestro	siniestro (GeoMedellín 2019-2025)	clase, gravedad, comuna, cámara cercana, lluvia
fact_clima_hora	hora	lluvia, temperatura, humedad, visibilidad
fact_snapshot_camara	cámara × captura	estado, brillo, ruta de la imagen, conteos YOLO opcionales
dim_camara	cámara	tipo, ubicación, comuna, enlace a foto, infracciones que detecta
dim_comuna · dim_cruce_semaforico · dim_fecha	—	polígonos y áreas, cruces, calendario con festivos y temporadas
stg_aforos, stg_velocidad_gt…	fuente	limpieza genérica

Vistas analíticas

Vista	Pregunta que responde
v_victimas_comuna_anio	¿Cuántas víctimas y muertos hubo por comuna y año, y con qué letalidad?
v_victimas_actor_franja	¿Qué actor vial resulta más afectado según la franja horaria y el tipo de día?
v_puntos_criticos	¿Qué celdas de unos 110 m concentran 20 o más víctimas, y qué tan lejos queda la cámara más cercana?
v_efecto_lluvia	¿Hay más víctimas por hora cuando llueve?
v_cobertura_camaras	¿Qué porcentaje de víctimas ocurre a 300 m o menos de una cámara?
v_camaras_cctv_siniestros	¿Cuántos siniestros 2019-2025 hubo alrededor de cada cámara CCTV?
v_incidente_con_victimas	Cada incidente con su número de víctimas y muertos.

Salidas adicionales: `data/reports/calidad_datos.md` (reporte legible), `data/clean/camaras.csv` e `incidentes_para_camaras.csv`, que se cargan en la página web de consulta de cámaras.

6. Cámaras CCTV y visión artificial

Módulo **etl/camaras.py**. Toma la lista de cámaras de dim_camara y, en cada ronda (por defecto cada 5 minutos):

Paso	Qué hace
1. Descarga	Pide la foto de cada cámara CCTV con enlace válido.
2. Control de calidad	error (HTTP distinto de 200 o sin imagen), sin_senal (menos de 8 KB), congelada (misma huella SHA-1 que la ronda anterior) u ok.
3. Día o noche	Brillo medio de la imagen; por debajo de 60 se marca como noche.
4. Guardado	Solo imágenes nuevas: data/camaras/snapshots/AAAA-MM-DD/<cámara>/HHMMSS.jpg.
5. Visión artificial (opcional)	Con conteo_vehiculos: true, YOLOv8n cuenta autos, motos, buses, camiones, bicicletas y personas por foto.
6. Registro	Una fila por cámara y ronda en Parquet particionado por fecha; vista fact_snapshot_camara en DuckDB.

El conteo de YOLO es un aforo propio por cámara y hora que se cruza con la siniestralidad mediante id_camara_cctv_cercana. Las cámaras no publican histórico ni video, así que la serie empieza el día en que se enciende el colector. La visión artificial viene desactivada y aún no se ha probado con el modelo real.

7. Operación

7.1 Instalación y comandos

```
python -m venv .venv && .venv\Scripts\activate
pip install -r requirements.txt

python -m etl run                      # extract + transform + load
python -m etl run --only victimas comunas camaras_cctv
python -m etl transform                # re-procesar sin descargar
python -m etl extract --force          # forzar descarga
python -m etl camaras --rondas 1       # una foto de cada cámara
python -m etl camaras                  # captura continua cada 5 min
pytest -q                              # 16 pruebas
```

7.2 Programación

`programar_windows.bat` crea dos tareas en el Programador de tareas de Windows: el ETL completo los lunes a las 6:15 a. m. y la captura de cámaras cada 5 minutos entre 6:00 a. m. y 10:00 p. m. Los registros quedan en data/etl.log y data/camaras.log.

7.3 Estructura del proyecto

```
medellin_movilidad_etl/
├── config.yaml            fuentes, rutas, caja de Medellín, parámetros
├── etl/
│   ├── __main__.py        línea de comandos
│   ├── common.py          configuración, HTTP con reintentos, manifiesto
│   ├── extract.py         MEData, ArcGIS, clima
│   ├── cleaning.py        texto, fechas, coordenadas, banderas
│   ├── geo.py             polígonos y punto en polígono
│   ├── transform.py       reglas por fuente y enriquecimiento
│   ├── load.py            Parquet, DuckDB, vistas, reporte
│   └── camaras.py         colector CCTV y visión artificial
├── tests/                 16 pruebas con muestras reales
└── data/                  raw · clean · quarantine · warehouse · reports
```

7.4 Configuración

Todo se ajusta en config.yaml sin tocar código: activar o desactivar datasets (comparendos y velocidad e intensidad vienen apagados por su tamaño), la caja envolvente de Medellín, las coordenadas comodín conocidas, el intervalo de captura y la visión artificial.

8. Pruebas

Grupo	Qué verifica
Unitarias	Reparación de texto (mojibake, \xNN, \r), coma decimal, nulos y categorías canónicas, distancia haversine.
Extremo a extremo	Con muestras reales de víctimas, incidentes, cámaras y comunas: sin duplicados, fechas válidas, categorías cerradas, coordenadas dentro de Medellín, comodín anulado, bodega con tablas y vistas.
Siniestralidad 2019-2025	Dos registros reales de la capa y cuatro casos de ruido sintético: radicado repetido, fecha imposible, punto en Envigado y escape hexadecimal.
Extracción sin red	Descarga incremental de MEData, paginación de ArcGIS y estados del colector de cámaras con respuestas simuladas.

9. Limitaciones y próximos pasos

- MEData publica víctimas hasta 2021 e incidentes hasta 2020; la capa de GeoMedellín cubre 2019-2025 pero no incluye víctimas individuales ni muertos.
- La extracción contra los servidores reales debe validarse en el equipo donde se ejecute; en el entorno de desarrollo se probó con respuestas simuladas.
- La visión artificial está implementada pero desactivada; falta validarla con imágenes nocturnas y calibrar el umbral de confianza.
- Siguiente paso sugerido: publicar la bodega en SQL Server o PostgreSQL y conectar el dashboard en vivo.

Licencia de los datos: CC BY-SA 4.0. Citar «Alcaldía de Medellín – MEData / GeoMedellín / SIMM».